

Data structure

1. ① Linear data structures organize elements sequentially, like a list, where each element has a unique predecessor and successor. For example arrays, linked lists and queues.

Non linear data structures don't follow a sequential order. Elements can have multiple predecessors, successors and forming hierarchical relationships. For examples Trees and Graphs. These structures offer more complex relationships among elements compared to the linear data structures.

⑥ $A(2:8, -4:1, 6:10, 3:7, -2:6)$

Here,

$$L_1 = (8-2)+1 = 7$$

$$L_2 = (1+4)+1 = 6$$

$$L_3 = (10-6)+1 = 5$$

$$L_4 = (7-3)+1 = 5$$

$$L_5 = (6+2)+1 = 9$$

④

$$E_1 = K_1 - 2 = 3 - 2 = 1$$

$$E_2 = K_2 + 4 = -1 + 4 = 3$$

$$E_3 = K_3 - 6 = 8 - 6 = 2$$

$$E_4 = K_4 - 3 = 4 - 3 = 1$$

$$E_5 = K_5 + 2 = 2 + 2 = 4$$

In row major order,

$$\text{Address} = \text{base} + w (((((E_1 L_2 + E_2) L_3 + E_3) L_4 + E_4) L_5 + E_5)))$$

$$= 450 + 4 (((((1 \times 6 + 3) 5 + 2) 5 + 1) 9 + 4)))$$

$$= 450 + 8512 = 8912$$

In column major order,

$$\text{Address} = \text{base} + w ((((E_5 L_4 + E_4) L_3 + E_3) L_2 + E_2) L_1 + E_1))$$

$$= 450 + 18064 = 18464$$

© Hence, $(A * (B + D) / E - F * (G + H / K))$

Symbol	Stack	Postfix
A	(	A
*	(*)	
(	(*(	
B		AB
+	(*(+)	
D		ABD
)	(*)	ABD+
/	(/	ABD+*
E		ABD+*E
-	(-)	ABD+*E/
F		ABD+*E/F
*	(-*)	
(	(-*()	
G		ABD+*E/FG

Symbol	Stack	Postfix
+	(-*(+	
H		ABD+*E/FGH
/	(-*(+/	
K		ABD+*E/FGHK
)	(-*	ABD+*E/FGHK/+
)		ABD+*E/FGHK/+*-

2.

- ② A stack overflow occurs when the call stack of a program exceeds its maximum allowed size. This typically happens due to a deep or infinite recursion, where functions keep calling themselves without reaching a base case. To minimize the risk of stack

overflow you can take several measures

1. Optimize Recursive functions:

If we are using recursion, make sure your recursive functions are optimized and have a clear base case. Avoid unnecessary recursive calls and make sure the recursive depth is reasonable.

2. Use Iterative Solutions:

In some cases, we can convert recursive solutions to iterative ones, which often use less stack space. This is especially true for tail-recursive functions, where the recursive call is the last operation in the function.

3. Increase Stack Size (if possible):

Some programming languages allow to increase the size of the call

stack. However, this might not be a viable solution in all cases, as it depends on language and system limitations.

4. Limit Recursion Depth:

Implement checks to limit the recursion depth to a reasonable level. This can help prevent excessive stack usage.

5. Usage an Explicit Stack:

If applicable, use an explicit stack data structure instead of relying on the call stack. This is particularly useful for tree traversal algorithms.

By applying these techniques, we can reduce the risk of stack overflow in our programs. However, it's crucial to understand the specific requirements and constraints of our application to choose most appropriate approach.

⑤

①

INFO
1 A
2 X
3 B
4 D
5 E
6 G
7 Y
8 C
9 F
10 Z
11 W
12

START → 5

AVAIL → 12

PRN

2
7
4
4
1
2
3
5
4
2
1

LINK

6
0
4
9
11
10
3
2
8
7
1

⑪

INFO
1
2 X
3 B
4 D
5
6 G
7 Y
8 C
9 F
10 Z
11
12

START → 6

AVAIL → 1

PRN

7
4
4
2
3
5
4
2

LINK

0
4
9
10
3
2
8
7

① Quicksort

66 33 40 22 53 88 60 11 80 20
 ↑
 Pivot

66 33 40 22 53 20 60 11 80 88
 i j

11 33 40 22 53 20 60 66 80 88
 i j

11 33 40 22 53 20 60 66 80 88
 i j

11 33 20 22 53 40 60 66 80 88
 i j

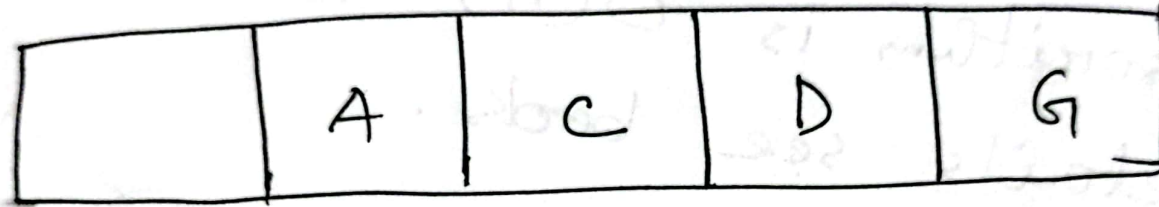
11 22 20 33 53 40 60 60 80 88
 i j

11 20 22 33 40 53 60 60 80 88
 i j

This is our Sorted array by
 Using Quicksort Algorithm.

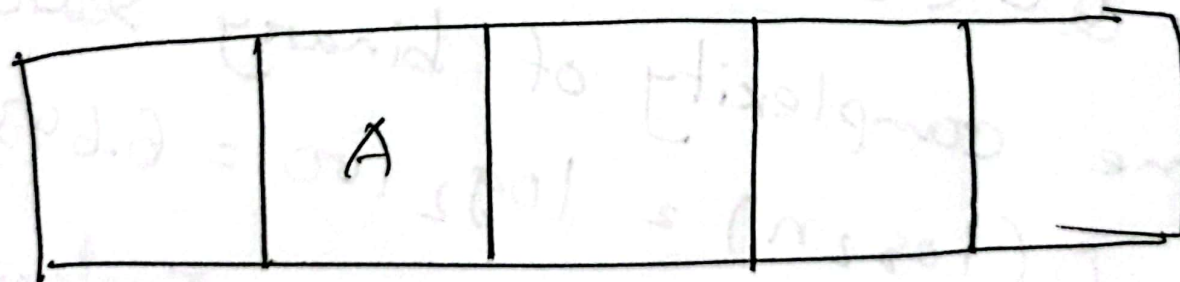
3.

① G is added to the right.



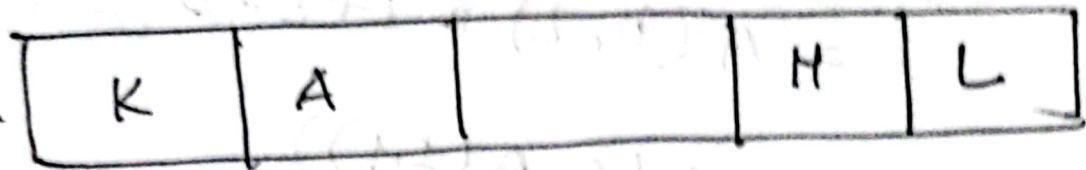
LEFT = 2, RIGHT = 5

② Three Letters on the right are deleted.



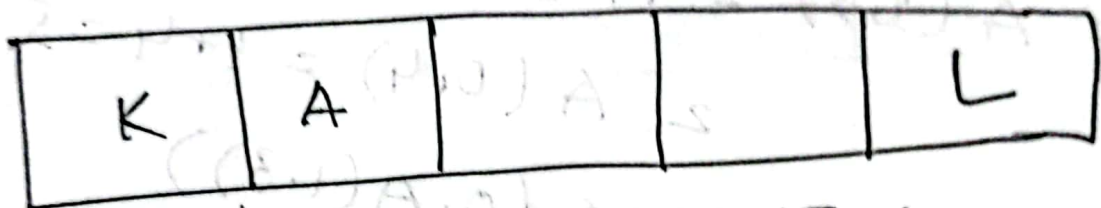
LEFT = 2, RIGHT = 2

⑪ K, L, M are added to the left



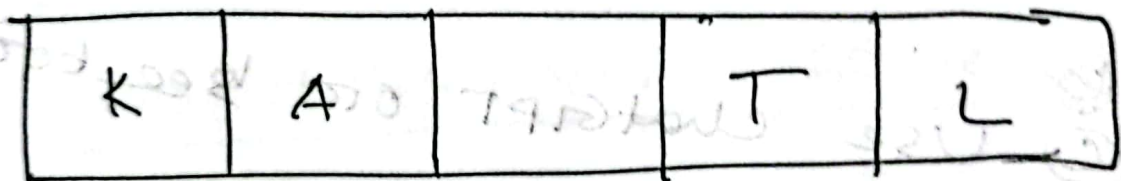
LEFT = 4, RIGHT = 2

⑫ One Letter Left is deleted,



LEFT = 3, RIGHT = 2

⑬ T is added to the left



LEFT = 4, RIGHT = 2

⑤ Here, $h(K) = K \bmod 7$

①

$$h(12) = 12 \bmod 7 = 5$$

$$h(26) = 26 \bmod 7 = 5$$

$$h(7) = 7 \bmod 7 = 0$$

$$h(23) = 23 \bmod 7 = 2$$

$$h(17) = 17 \bmod 7 = 3$$

0	7	Using Chaining Method
1		
2	23	
3	17	
4		
5	12	→ 26 x
6		

① We know,

$$h'(K) = [h(K) + i] \bmod m$$

$$h(12) = 12 \bmod 7 = 5$$

$$h(26) = 26 \bmod 7 = 5$$

$$h'(26) = [5 + 0] \bmod 7$$

$$= 5$$

$$h'(26) = [5 + 1] \bmod 7$$

$$= 6$$

$$h(7) = 7 \bmod 7$$

$$h(23) = 23 \bmod 7$$

$$h(17) = 17 \bmod 7$$

	0	7
	1	
	2	23
	3	17
	4	
	5	12
	6	26

→ Using
Linear Probing
Method

(iii) Here,

$$h(K) = K \bmod 7$$

$$h'(K) = 5 - (K \bmod 5)$$

$$h''(K) = [h(K) + i * h'(K)] \bmod 7$$

$$h(12) = 12 \bmod 7 = 5$$

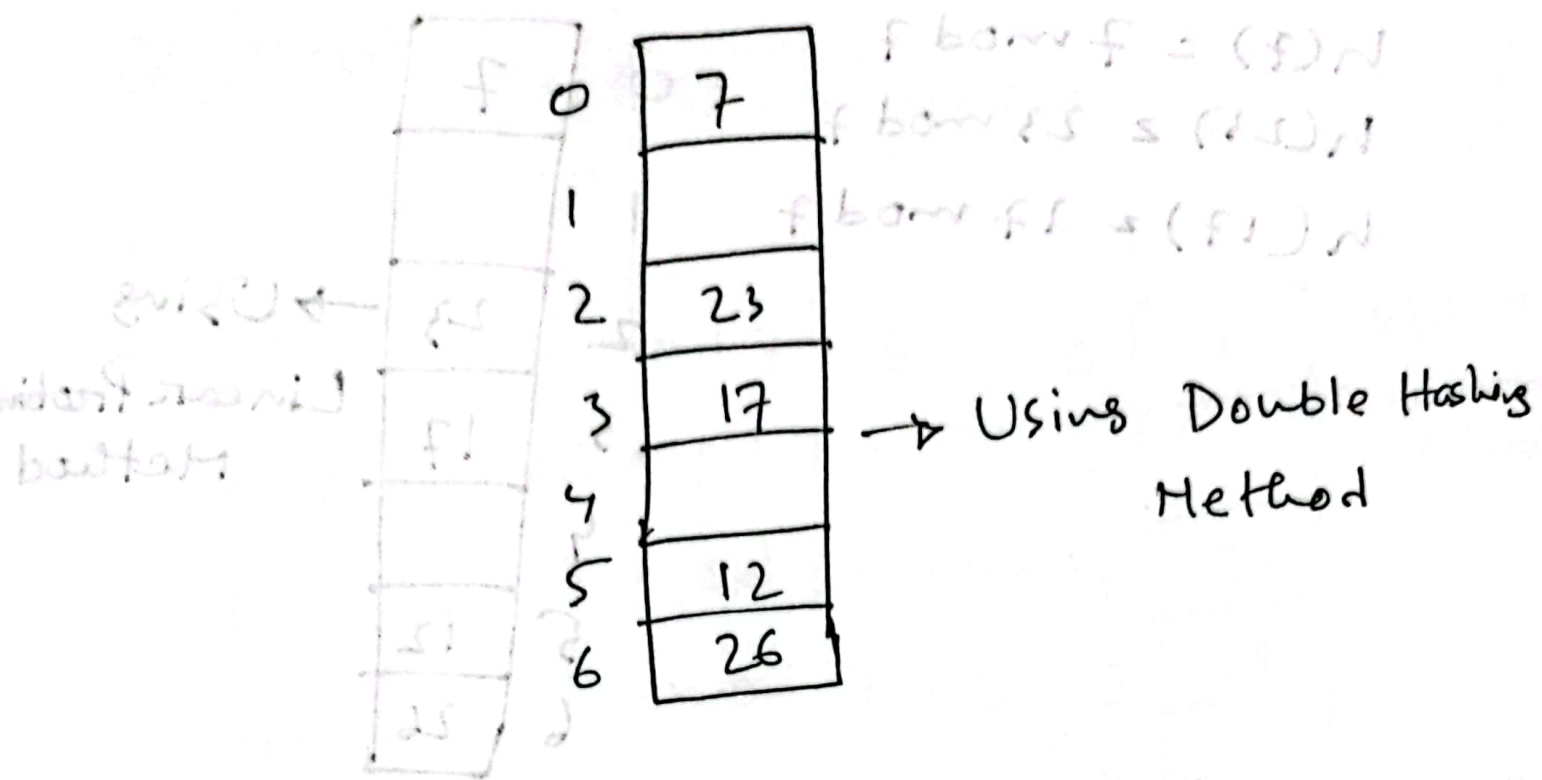
$$h(26) = 26 \bmod 7 = 5$$

$$h'(26) = 5 - (26 \bmod 5) = 4$$

$$h''(K) = [5 + 0] \bmod 7 = 5$$

$$= [5 + 1 * 4] \bmod 7 = 9 \bmod 7$$

$$= [5 + 2 * 4] \bmod 7 = 6$$



© Use ChatGPT or see Book for this prob.

Σ:

②

$$A(1,4) = A(0, A(1,3))$$

$$A(1,3) = A(0, A(1,2))$$

$$A(1,2) = A(0, A(1,1))$$

$$A(1,1) = A(0, A(1,0))$$

$$A(1,0) = A(0,1)$$

$$A(0,1) = 1+1 = 2$$

$$A(1,0) = 2$$

$$A(1,1) = A(0, A(1,0))$$

$$= A(0, 2) = 1+2 = 3$$

$$A(1,2) = A(0, A(1,1))$$

$$= A(0, 3) = 1+3 = 4$$

$$A(1,3) = A(0, A(1,2))$$

$$= A(0, 4) = 1+4 = 5$$

$$A(1,4) = A(0, A(1,3))$$

$$= A(0, 5) = 1+5 = 6$$

⑥ Use ChatGPT or see book for this problem.

4.

② 44, 33, 11, 55, 77, 44, 40, 90, 60

Pass 1:

33, 44 11, 55

44, 77

40, 90 60

Pass 2:

11, 33, 44, 55

40, 44, 77, 90 60

Pass 3:

11, 33, 40, 44, 44, 55, 70, 90 60

Pass 4:

11, 33, 40, 44, 44, 55, 60, 70, 90

↑
Sorted

③ see book for this problem

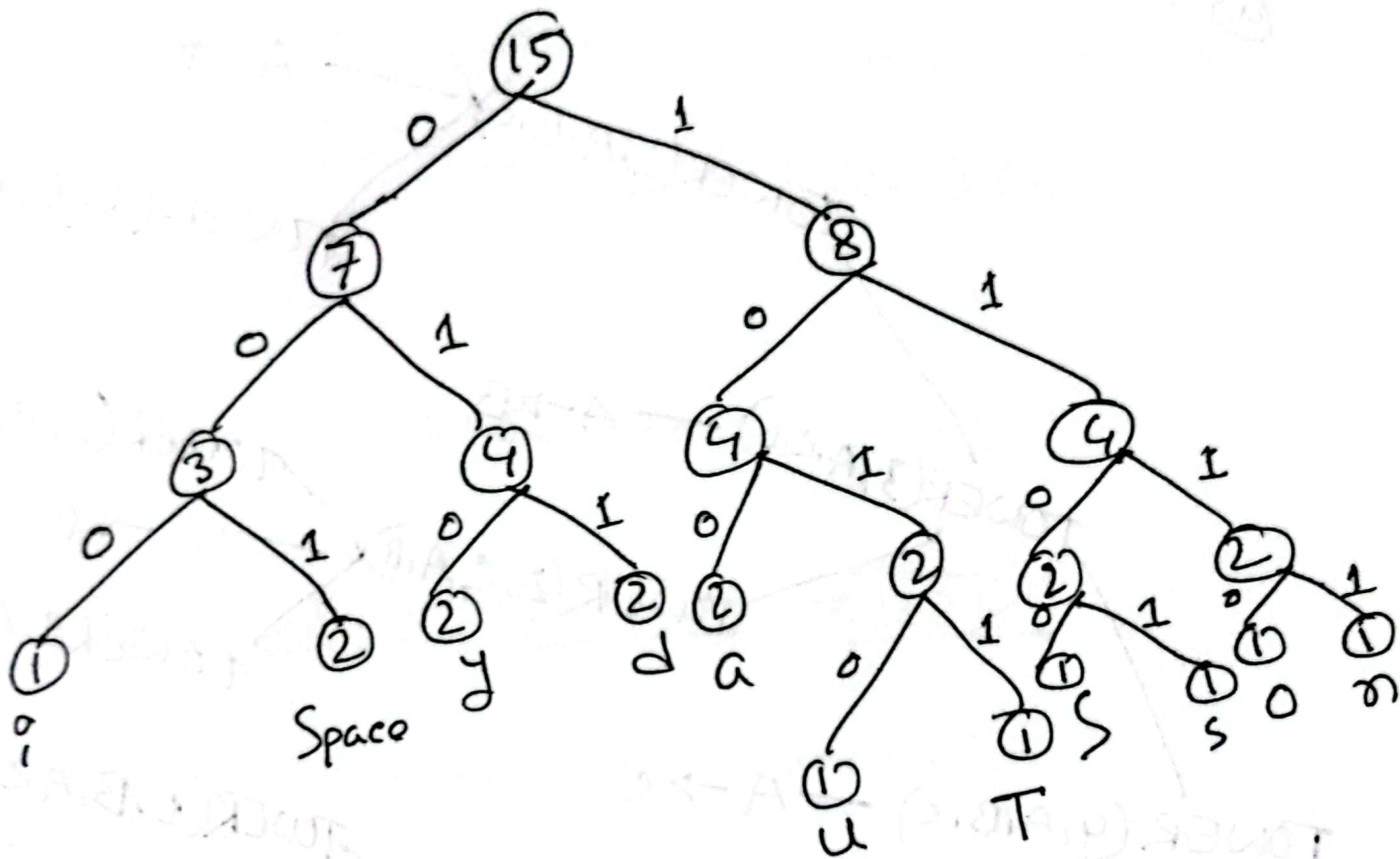
© "Today is Sunday"

Here total bits = $15 \times 8 = 120$ bits

<u>character</u>	<u>Count</u>	<u>bits</u>	
a	2	100	→ 6
d	2	011	→ 6
i	1	000	→ 3
n	1	1111	→ 4
o	1	1110	→ 4
s (small)	1	1101	→ 4
S	1	1100	→ 4
T	1	1011	→ 4
u	1	1010	→ 4
y	2	010	→ 6
Space	2	001	→ 6
		<u>39 bits</u>	<u>51 bits</u>

→ $8 \times 11 = 88$ bits

Using Huffman coding we get,



∴ Total bits set after Using Huffman Coding method = $88 + 39 + 51 = 178$ bits which is greater than 120. So, the effectiveness of this method is zero. Huffman coding method isn't useful for this ~~method~~ sentence.

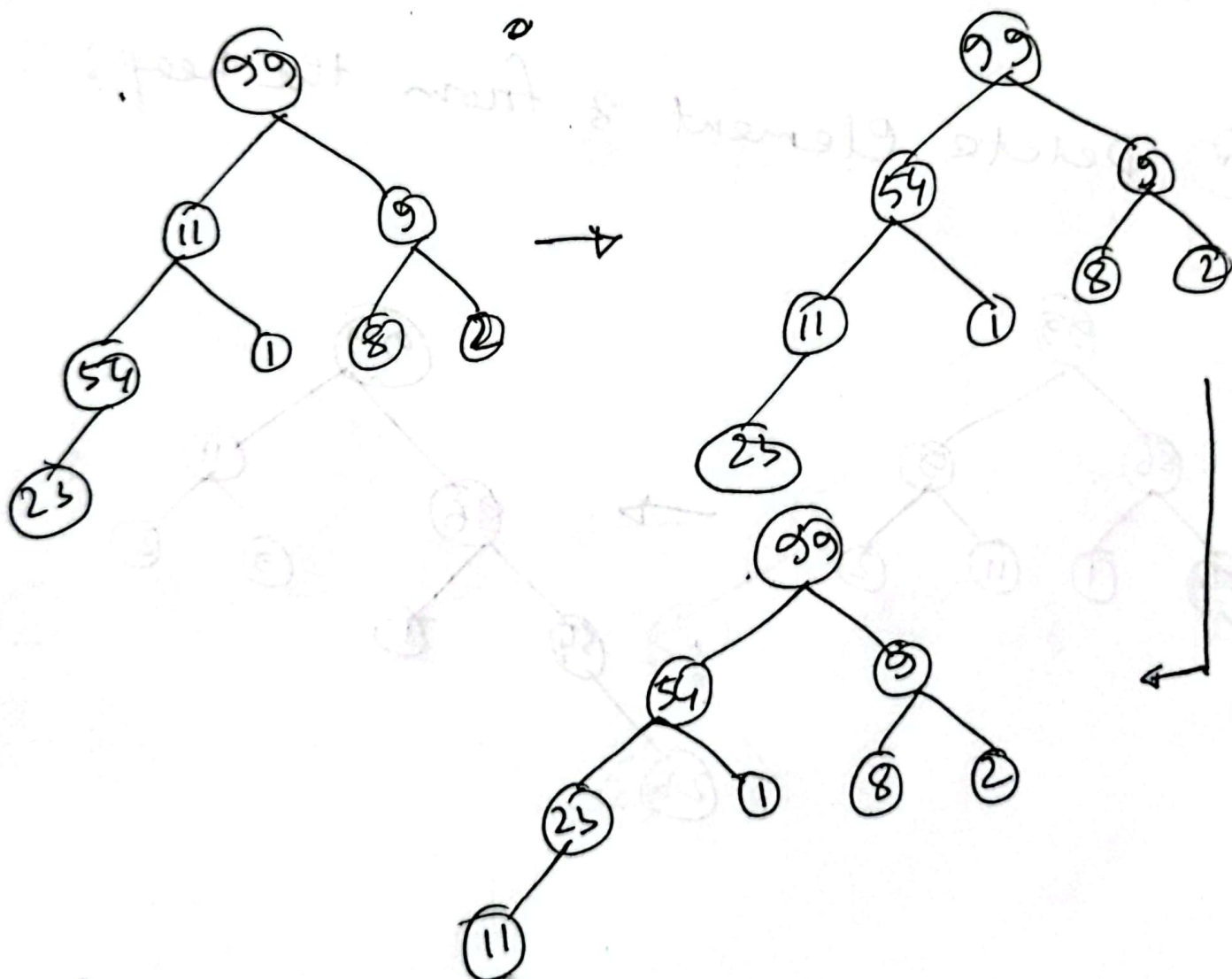
6

(a) use chatGPT for this ques.

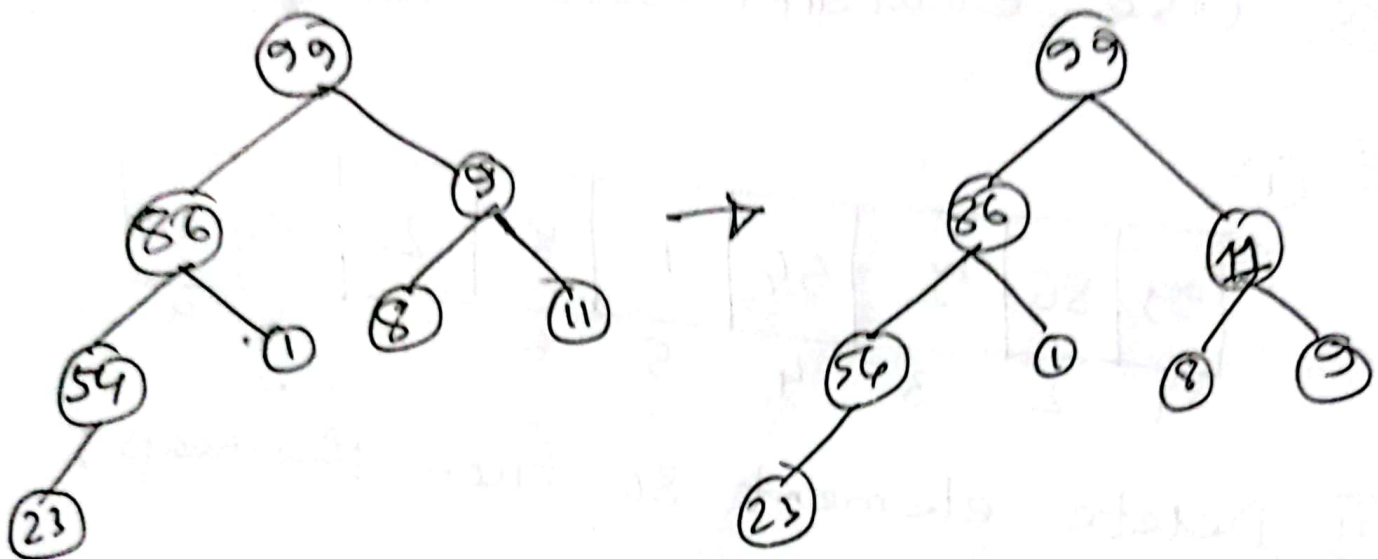
(5) (1)

99	86	9	54	1	8	2	23	11
1	2	3	4	5	6	7	8	9

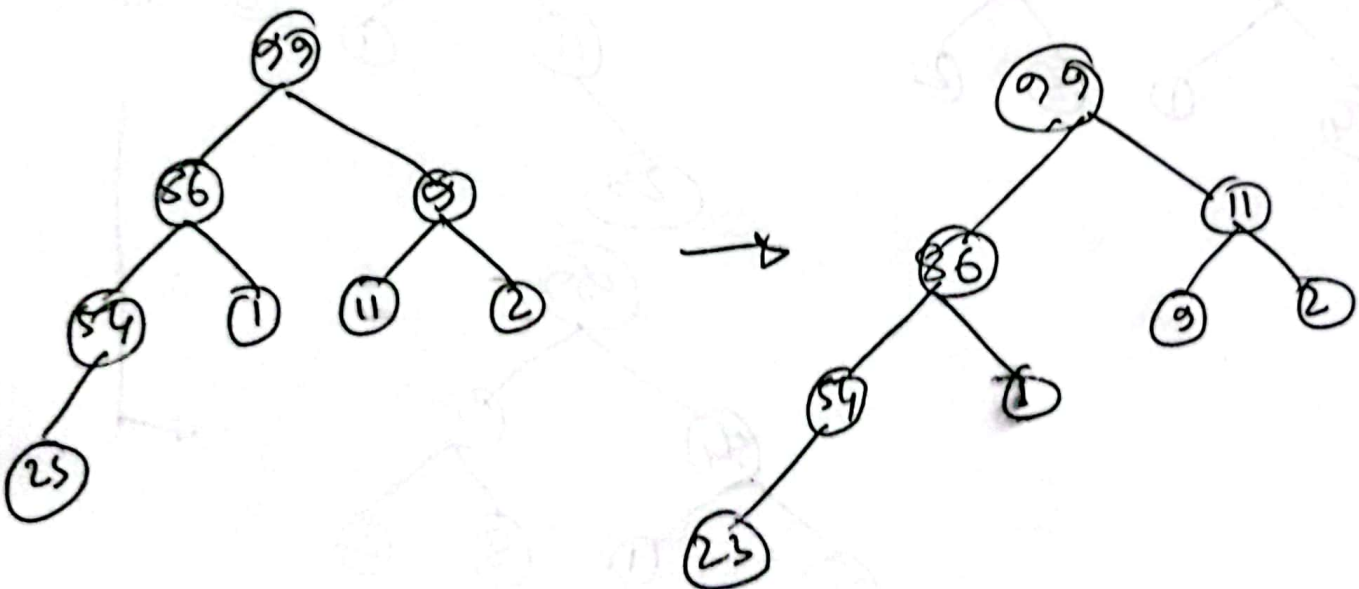
(ii) Delete element 86 from the heap:



⑬ Delete element 2 from the heap:

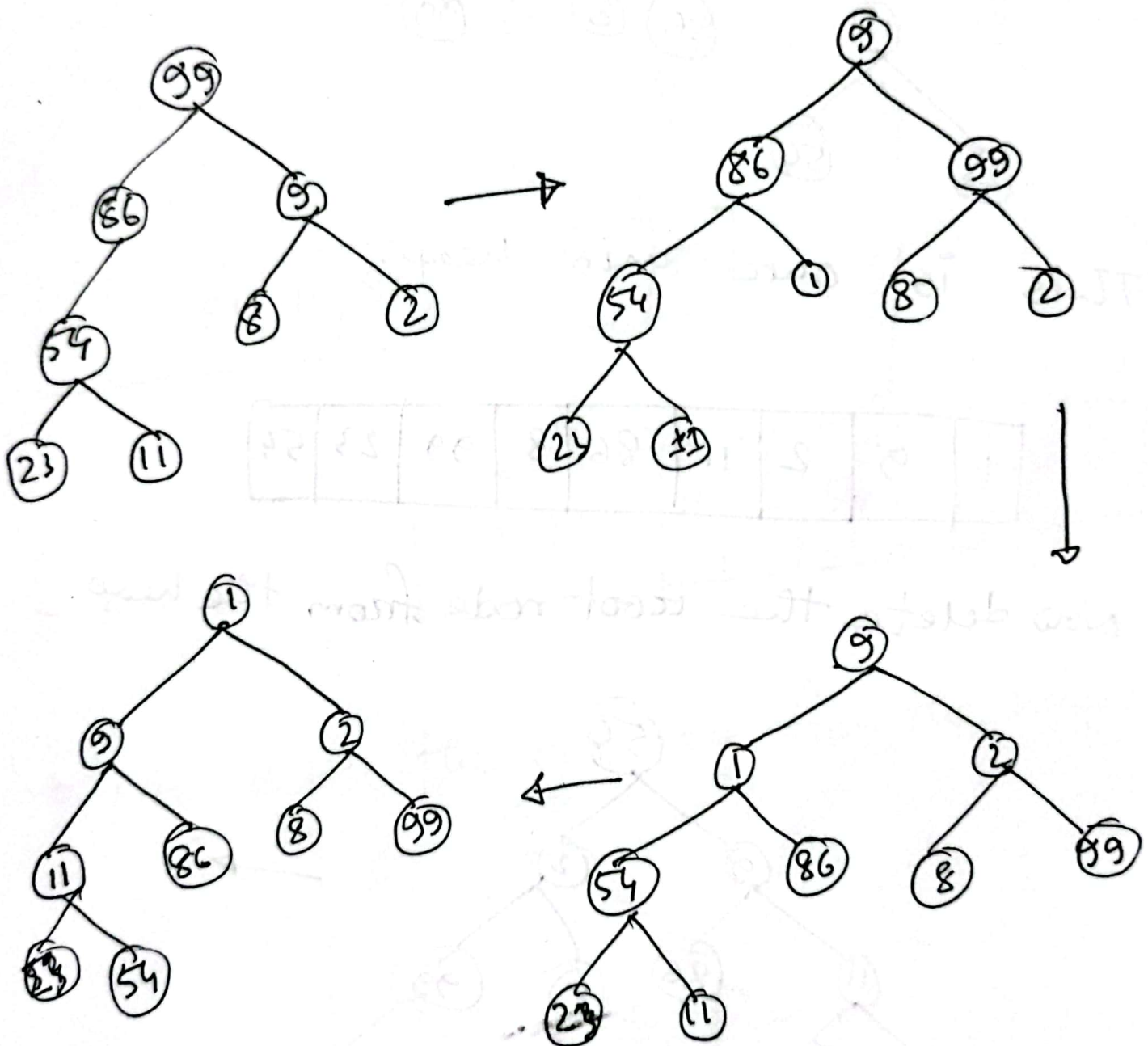


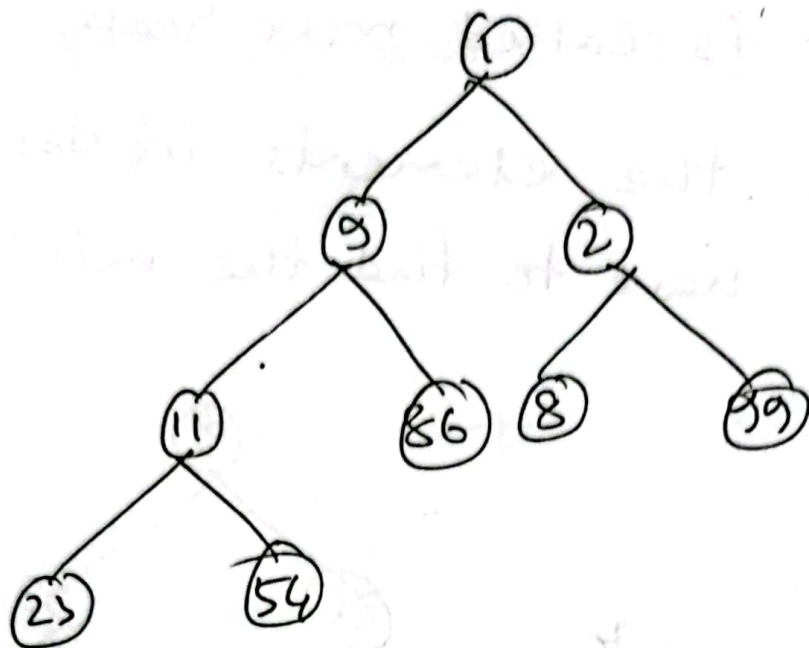
⑭ Delete element 8 from the heap:



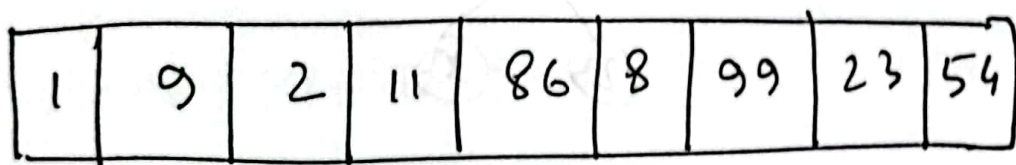
⑦ This heap is called max heap.

. For sorting the elements in descending order we have to find the min heap first.

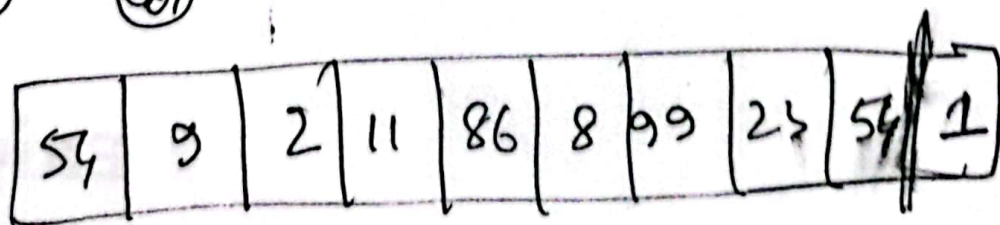
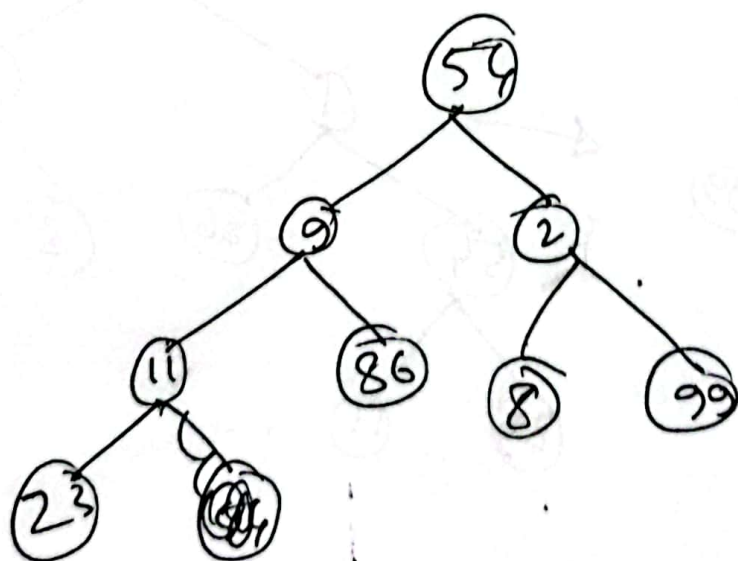


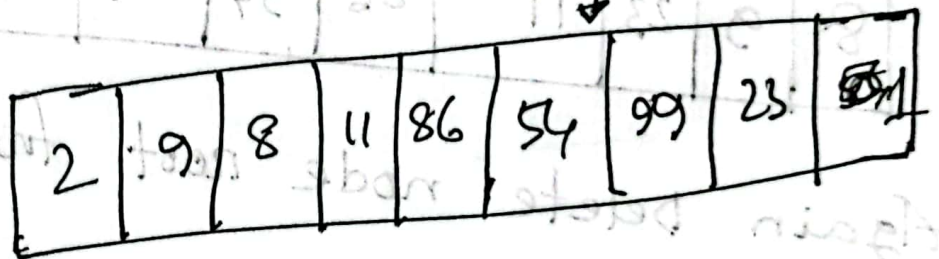
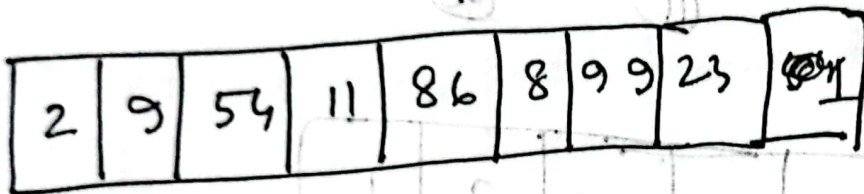
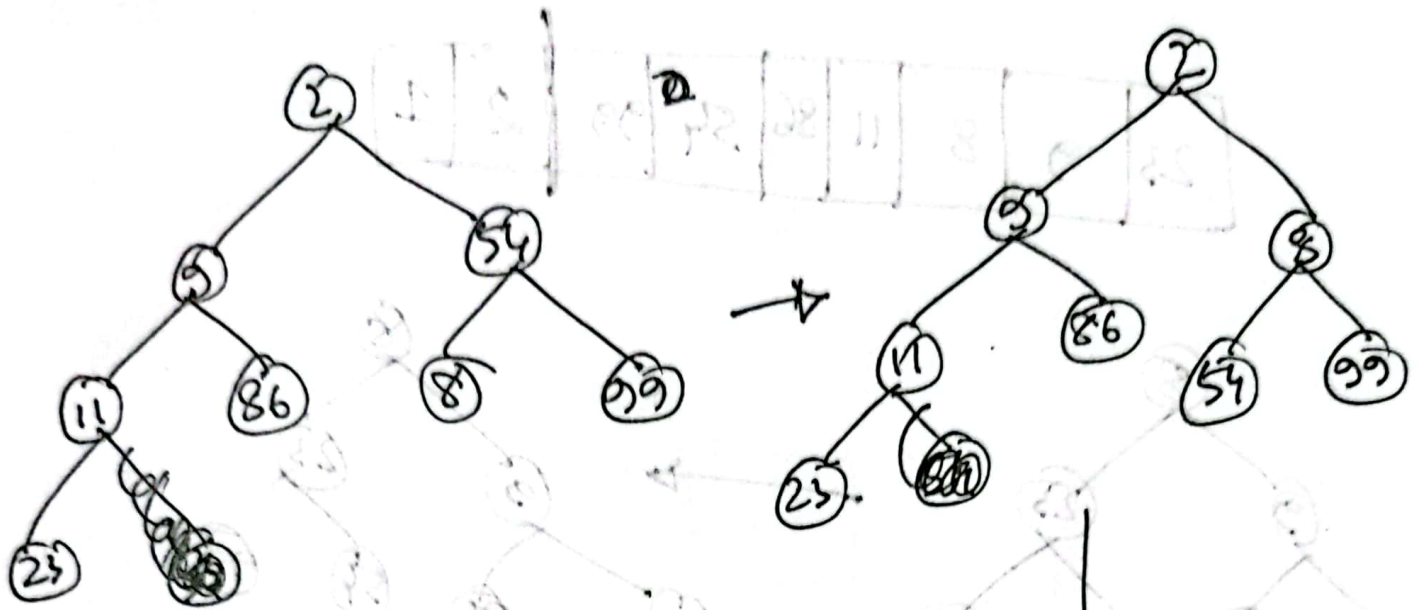


1 This is our min heap.

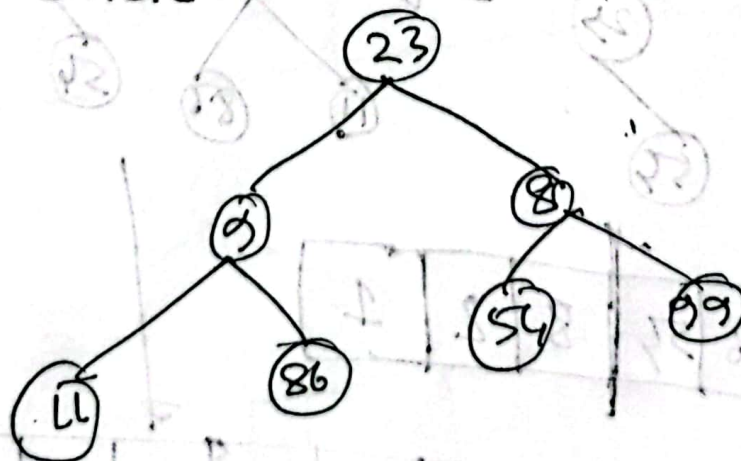


Now delete the root node from the heap.

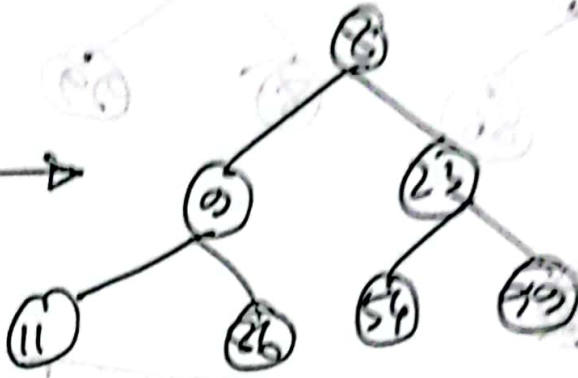
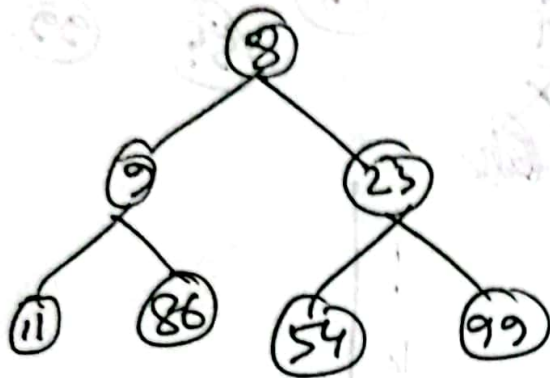




Now delete the root node from the heap

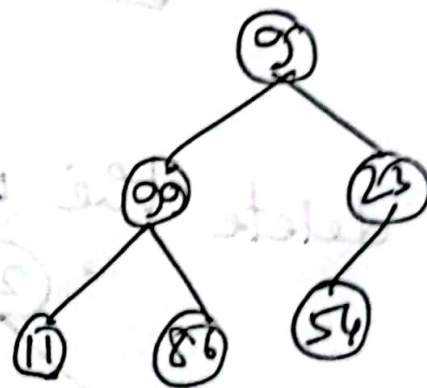
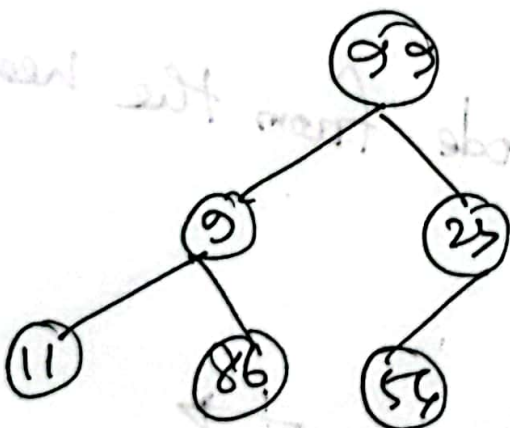


23	9	8	11	86	54	99	2	1
----	---	---	----	----	----	----	---	---



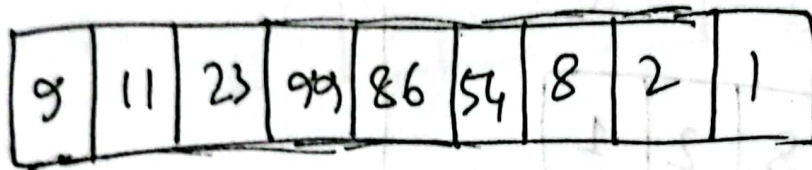
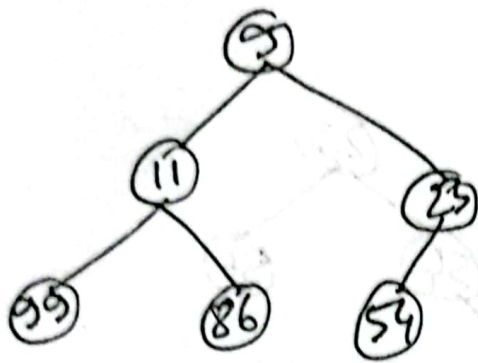
8	9	23	11	86	54	99	2	1
---	---	----	----	----	----	----	---	---

Again Delete node root from the heap.

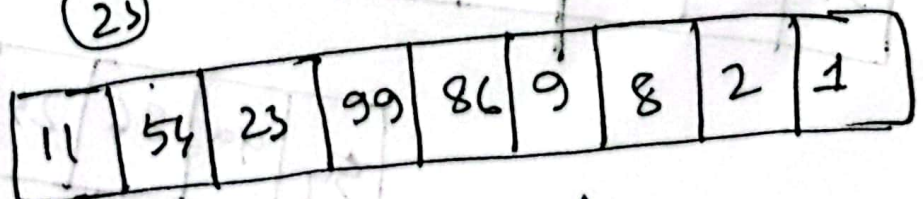
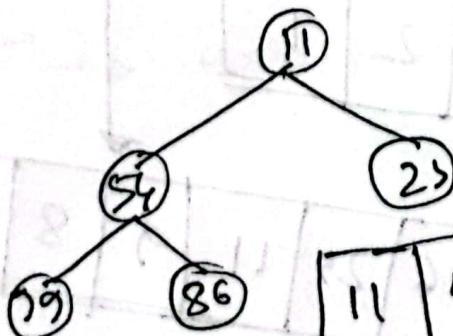
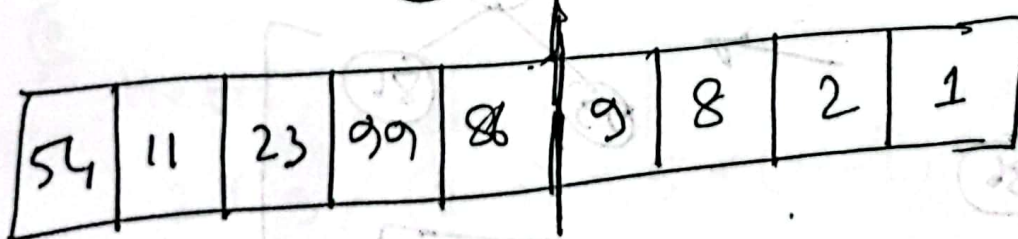
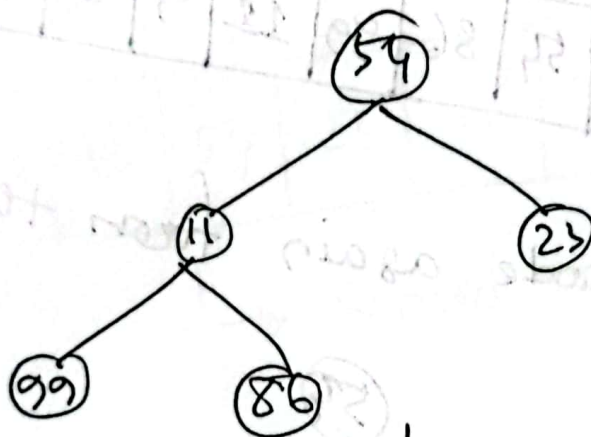


99	9	23	11	86	54	8	2	1
----	---	----	----	----	----	---	---	---

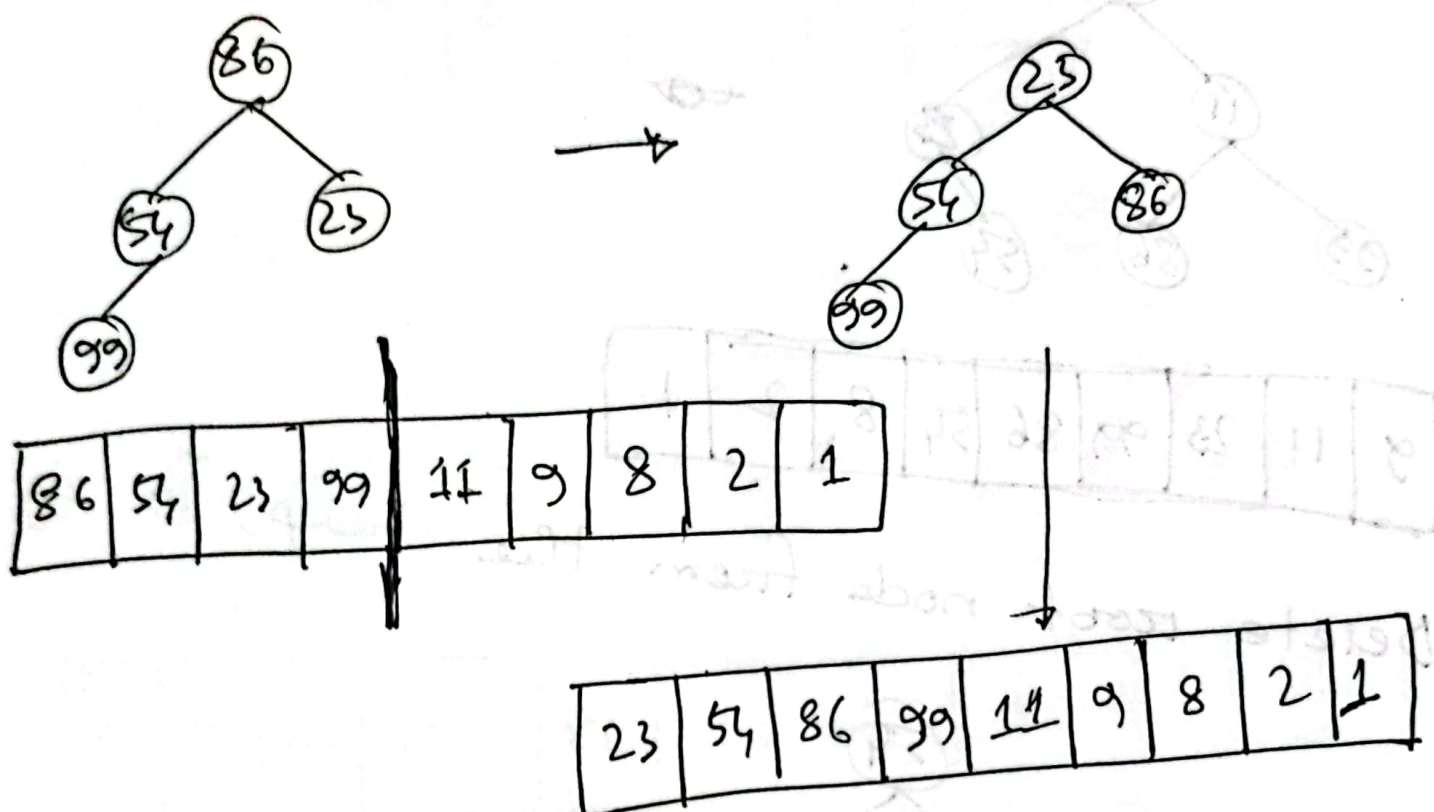
9	99	23	11	86	54	8	2	1
---	----	----	----	----	----	---	---	---



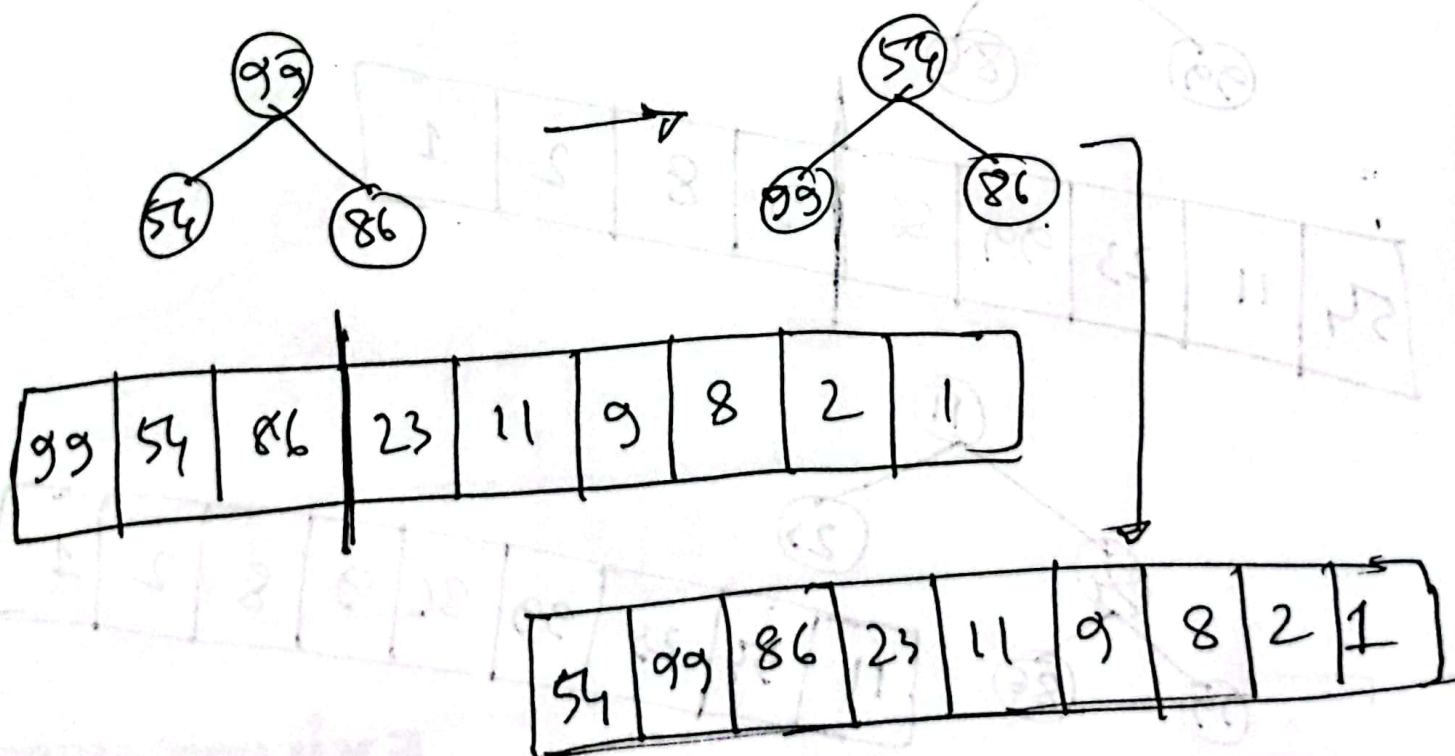
Delete root node from the heap.



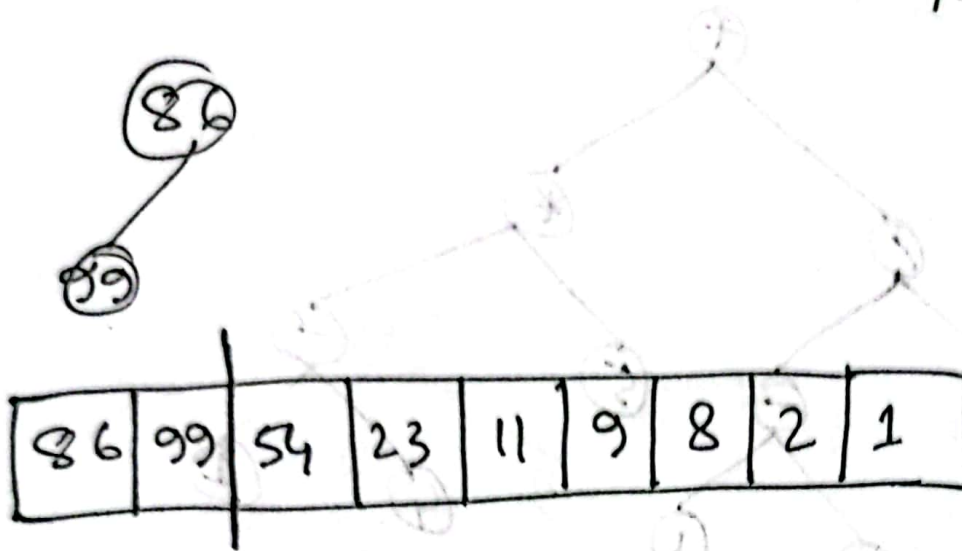
Also again the root node from the heap,



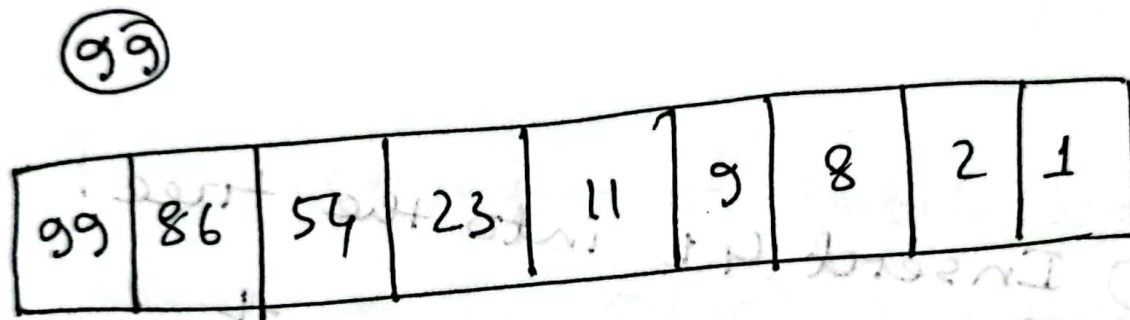
Delete the root node again from the heap.



Again delete the root node from the heap.



At last again delete the root node from heap.



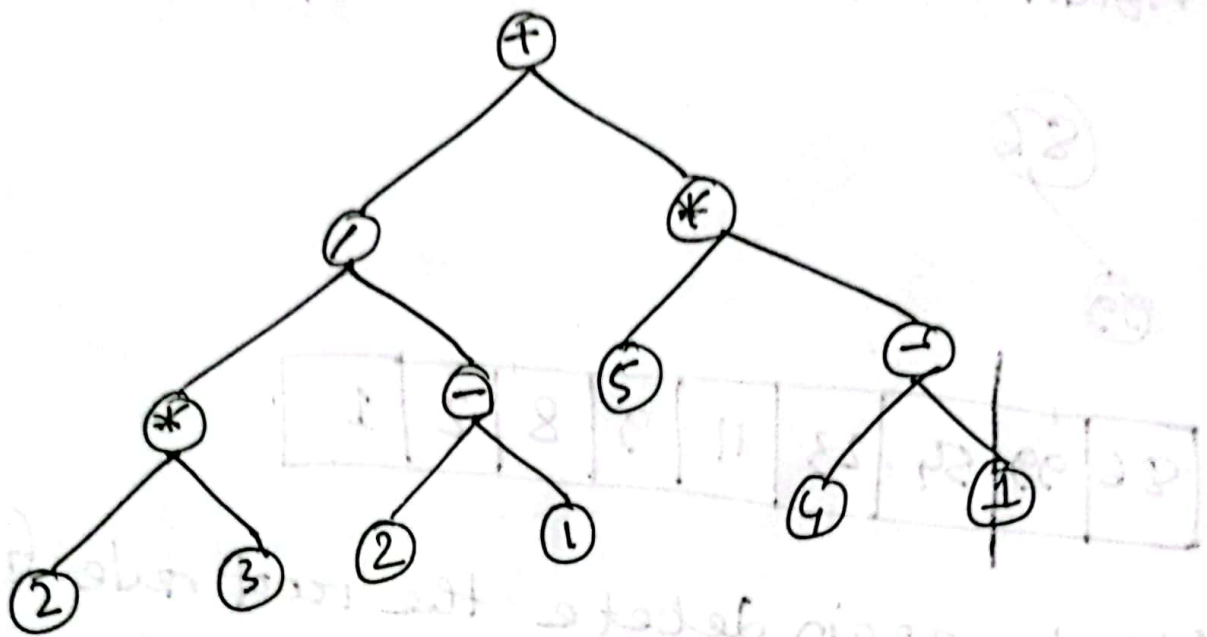
Sorted the array in descending order.

(c)

Here,

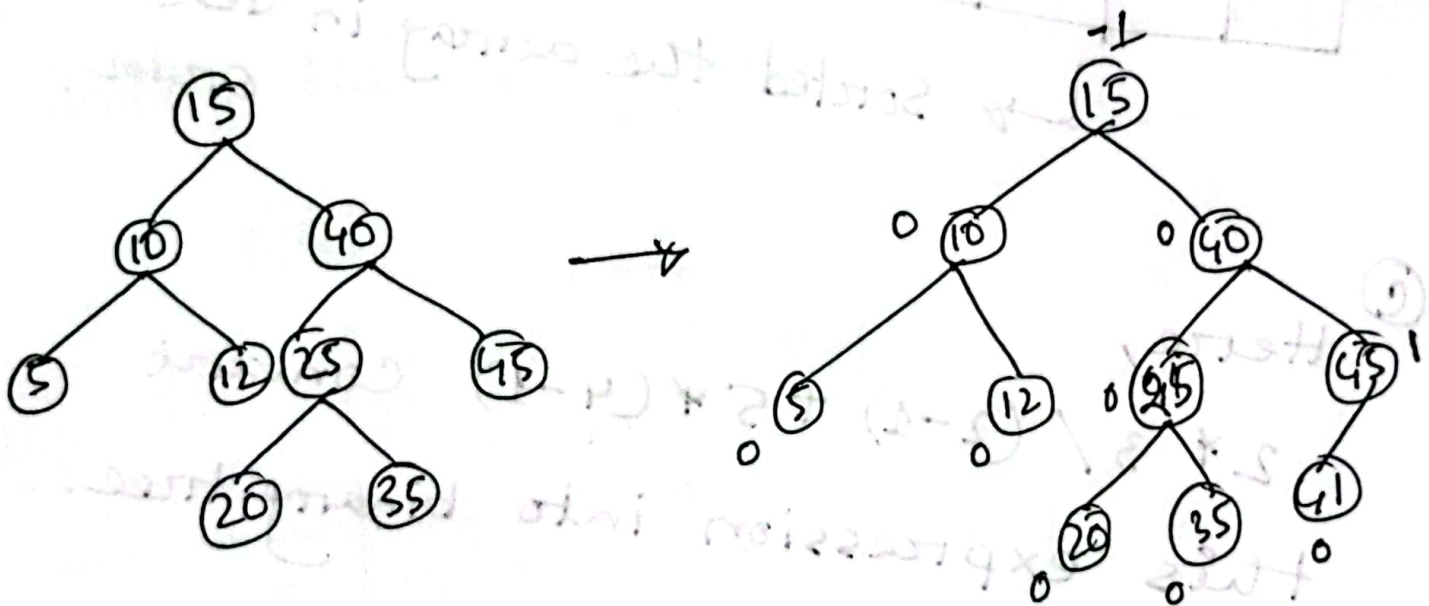
$2 * 3 / (2 - 1) + 5 * (4 - 1)$ Convert

this expression into binary tree.

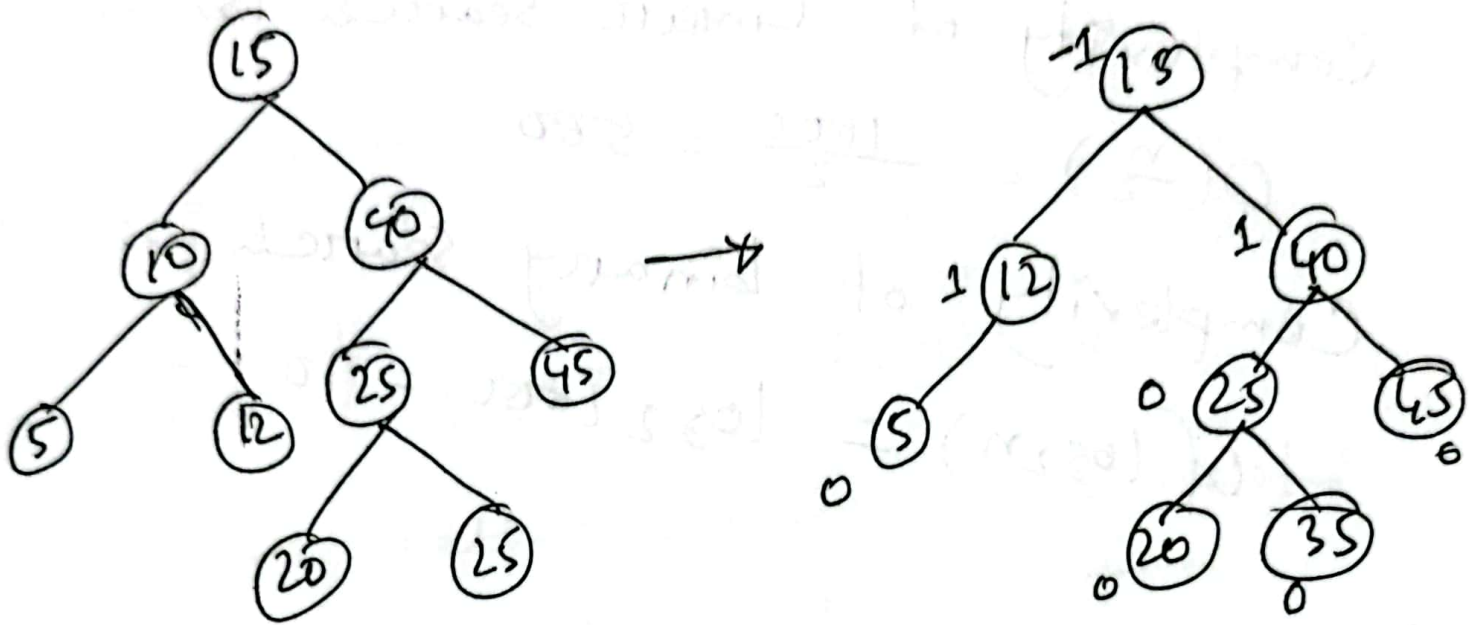


7.

① Insert 41 into the tree,



⑪ Delete 10 from the original tree,



⑫ Time complexity of bubble sort Algorithm is $O(n^2)$. For more details see books.

⑬ For,

$$n = 160$$

Time complexity of Linear Search,

$$O\left(\frac{n}{2}\right) = \frac{160}{2} = 80$$

Time complexity of binary Search,

$$O(\log_2 n) = \log_2 160 = 6.643$$

⑪ For $n = 1600$

Complexity of Linear Search is,

$$O\left(\frac{n}{2}\right) = \frac{1600}{2} = 800$$

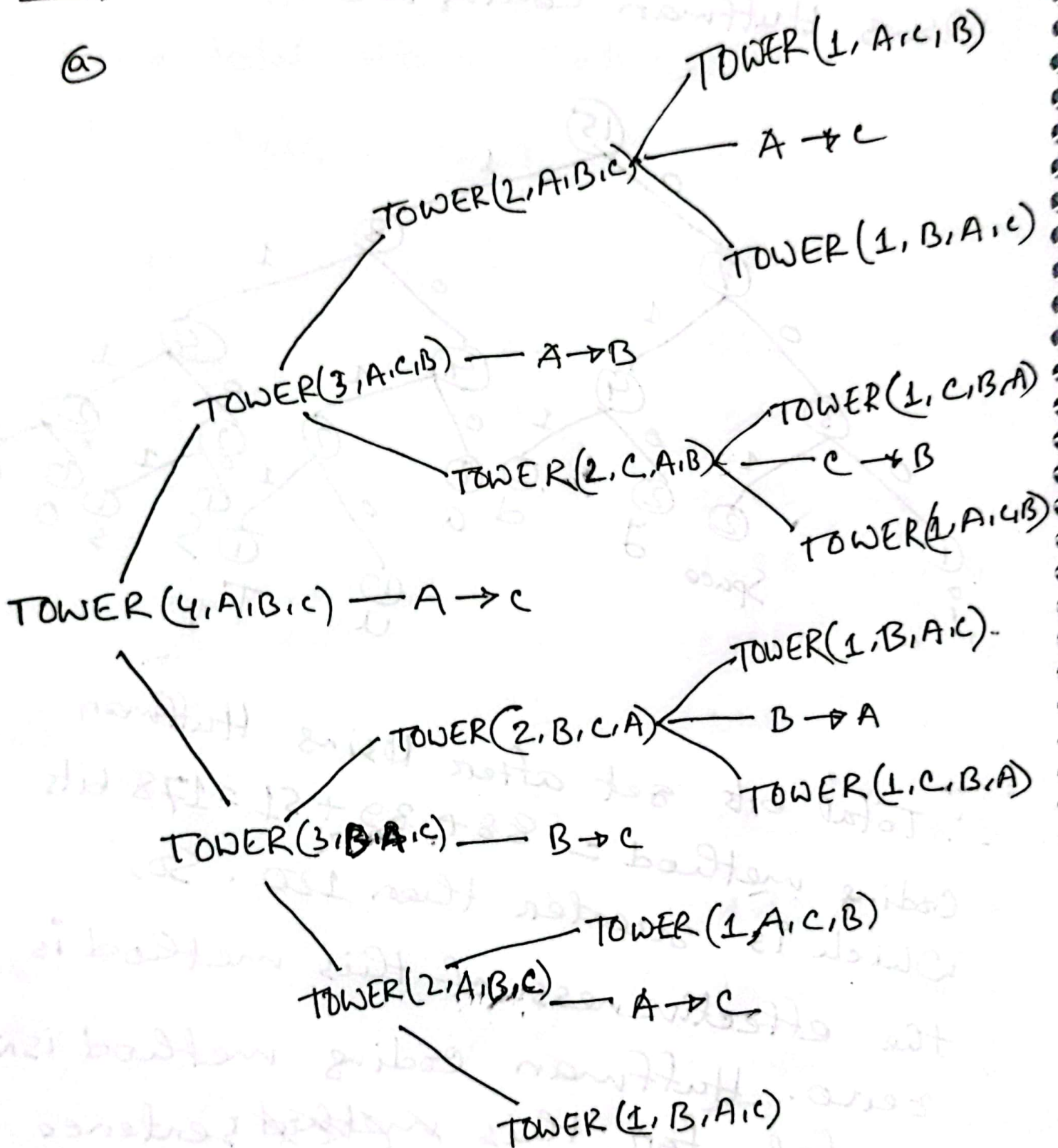
Complexity of binary search is,

$$O(\log_2 n) = \log_2 1600 = 10$$

3:

... to the right.

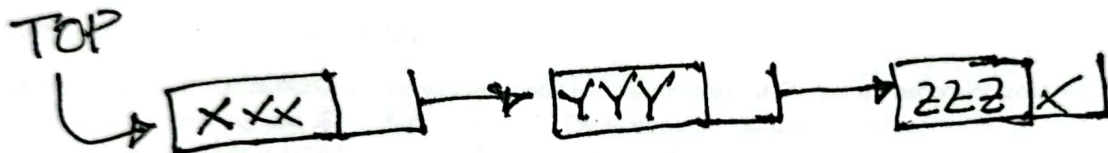
6



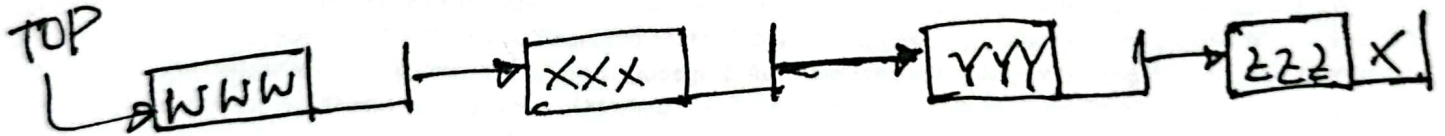
⑥ $O(n+k)$ complexity Algorithm is merging Algorithm. For more details see book Page 506.

⑦

Let a stack is,



STACK after a PUSH operation,



STACK after a POP operation,

